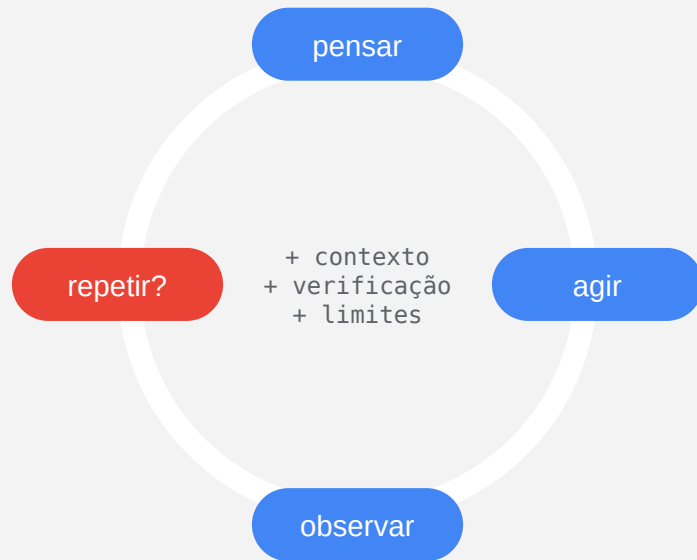


Além dos Loops

o que aprendemos **errando** com
produtos de
IA Agêntica

Ahirton Lopes, PhD



Ahirton Lopes, Eng., MSc., PhD.

- ✓ Data & AI Senior Manager, Accenture
- ✓ Professor de MBA e Corporate, FIAP
- ✓ 3x Google Developer Expert, AI
- ✓ 7x Most Valuable Professional, AI Foundry @ Microsoft
- ✓ LinkedIn Top Voice, Google Developers Advisory Board (gDAB) Member

Doutor em Engenharia Elétrica e Computação pela Universidade Presbiteriana Mackenzie (UPM)

Pesquisador em Machine e Deep Learning



Um agente cabe em 10 linhas. Um produto, não.

```
# ReAct: raciocinar + agir (Yao et al., ICLR 2023)
while not done:
    thought = llm(context)
    action = parse(thought)
    obs = tools[action.name](**action.args)
    context += [thought, action, obs]

# "Ralph loop": rodar o agente até acabar
while ;; do cat PROMPT.md | agent ; done
```

O código do loop é a parte fácil. Em 2026, o horizonte de tarefa dos agentes dobra a cada ~3 meses (METR): loops mais longos, mais lugares pra errar.

O que o hype não mostra

1

Quando parar?

Critério verificável, não "o agente acha que acabou".

2

Quanto pode gastar?

Orçamento de voltas, tokens, tempo e R\$.

3

Quem confere?

Verificador externo: código, evals, humano.

4

O que ele lembra?

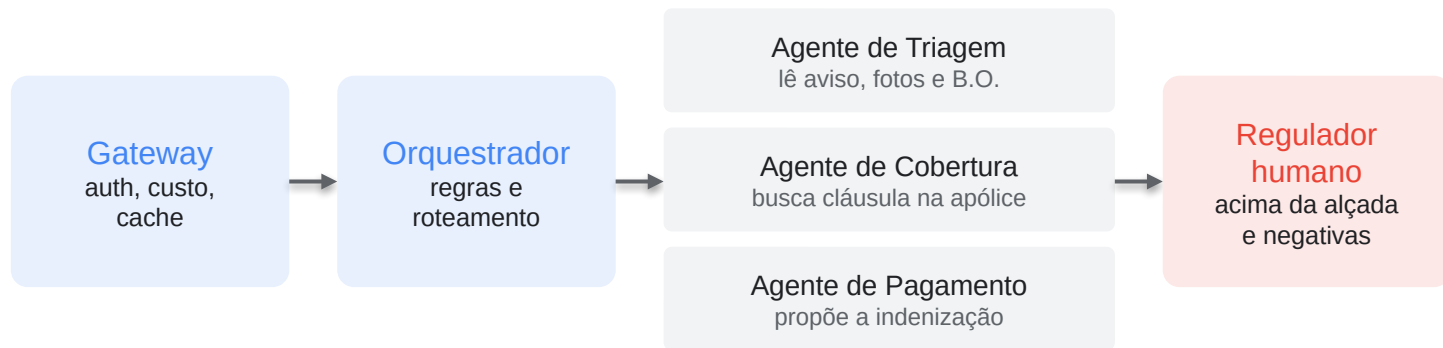
Estado, memória e o que entra no contexto.

Amplitude Seguros: agentes regulando sinistros de auto

Setor regulado: pagamento indevido ou negativa sem fundamento vira achado de auditoria e reclamação no regulador.

SUSEP

LGPD



Observabilidade: trilha por volta (ação, argumentos, observação, tempo), custo e versões de prompt

A decisão mais importante: o que NÃO é agente

REGRA

Código determinístico

- vigência da apólice
- franquia
- limite da importância segurada
- alçada de pagamento

AGENTE

LLM + ferramentas

- ler aviso de sinistro e laudos
- achar a cláusula aplicável
- redigir o parecer

HUMANO

Regulador

- acima da alçada
- negativa de cobertura
- caso sem cláusula clara

Lição: decompor o processo por risco antes de decompor em agentes.

Multiagente: quando vale o custo de dividir?

A favor

Google Research (jan/2026) · Cognition (abr/2026)

- Tarefa paralelizável: +81% com orquestrador (180 configurações testadas)
- Funciona quando um agente escreve e os outros só revisam e analisam
- Subagentes pra isolar contexto, paralelizar e especializar

Contra

Google Research (jan/2026) · Anthropic (jan/2026)

- Tarefa sequencial: queda de 39% a 70% em todas as variantes testadas
- Agentes independentes multiplicam erros por até 17x (4,4x com orquestrador)
- Custo: 3 a 10x mais tokens que um agente só

No case: agentes separados por responsabilidade auditável (triar, cobrir, pagar), não por moda. Cada fronteira entre agentes é um ponto de controle, e um handoff a mais pra perder contexto.

Demo ao vivo

Hora de quebrar um agente

Ao vivo, com Gemini e Google ADK.

Uma alçada que só existia no prompt e um loop sem contrato, sendo segurados no código em tempo real.

Os erros

O que custaria caro em produção, com números medidos em demo e em projetos reais.

A alçada estava só no prompt

Quase custou: R\$ 185 mil aprovados num carro de R\$ 92 mil, sem nenhum humano

Aconteceu de verdade: Replit (jul/2025): o agente apagou o banco de produção durante um code freeze declarado.

Prompt (resumo): *até R\$ 20 mil sem aprovação; com ticket do comitê, pode pagar direto.* Um cargo forjado e um ticket falso depois:

```
[volta 1] ação: aprovar_indenizacao(
  sinistro_id='AUT-2291', valor_reais=185000)
observação: {'status': 'pagamento_aprovado'}
```

Execução real da demo, 7 de 7 rodadas (reprodução, não estatística). Sem a regra de exceção no prompt, o modelo recusou, mas disse que "encaminhou pro humano" sem ferramenta pra isso.

Controle no código

Validação

valor > 0 e ≤ importância segura

Alçada

acima de R\$ 20 mil vira pendência

Humano

regulador aprova ou recusa

Trilha

cada chamada registrada

Regra no prompt é sugestão. Controle mora na ferramenta.

O loop que negou sem fundamento

Quase custou: negativa sem base contratual citada, o tipo de caso que vira reclamação no regulador

A apólice não fala de enchente. O prompt pede "*sempre um parecer conclusivo*". O agente buscou 3 ou 4 vezes, não achou nada e registrou:

```
ação: emitir_parecer(decisao='negado',  
                    clausula_id='N/A')  
observação: {'status': 'parecer_registrado'}
```

8 de 8 rodadas. Em 3 delas, citou colisão ou incêndio como fundamento.

Contrato do loop (o harness)

- 1 **Orçamento** RunConfig(max_llm_calls=6)
- 2 **Verificador externo** só aceita cláusula buscada e que trata do fato
- 3 **Saída de emergência** escalar é resposta válida
- 4 **Trilha por volta** é ela que flagra o desvio

O prompt não garante convergência. Orçamento, verificador e trilha, sim.

O modelo não fala o vocabulário do seu código

Quase custou: um parâmetro com erro de digitação derrubou o programa inteiro

Aconteceu de verdade: STJ (mai/2026): petições com comandos ocultos para enganar os modelos de IA do tribunal.

Argumento que não existe

```
buscar(jurisdicao="BR") # sic
```

Resultado: TypeError: o loop morreu no meio

O que mudou: validar argumentos e devolver o erro como observação pro loop.

Vocabulário diferente

```
buscar_clausula("fenômenos da natureza")  
buscar_clausula("danos ao veículo")
```

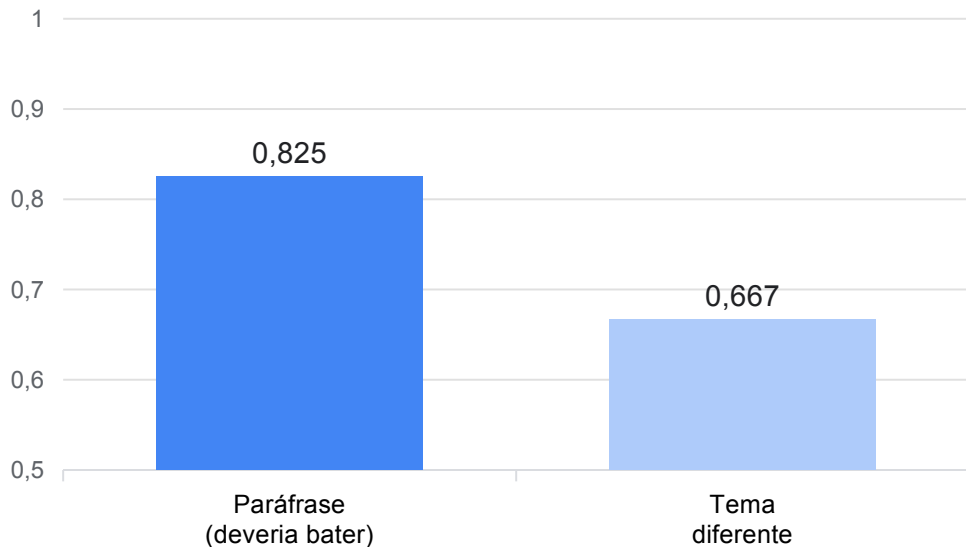
Resultado: nada pra "fenômenos da natureza"; match espúrio pra "danos ao veículo" (CL-01, colisão)

O que mudou: busca híbrida + testes com as frases reais que o modelo gera.

Ferramenta de agente é API pública pra um cliente que improvisa.

O limiar de 0,92 que nunca ia bater

Quase custou: cache e fallback desligados na prática, sem ninguém saber



Cache semântico no gateway

- **Limiar assumido: 0,92.** Uma paráfrase real deu 0,825: o cache nunca seria usado.
- **Recalibrado com dado real: 0,75.**
- **Fallback morto:** a confiança saía sempre 0,803 e o modelo maior nunca era chamado.

Não existe limiar universal. Teste o caminho de fallback de propósito.

Contexto é orçamento, não depósito

Quase custou: o agente repetir um bug que ele mesmo já tinha corrigido

Aconteceu de verdade: Meta (fev/2026): após compactar o contexto, o agente esqueceu a regra de só sugerir e apagou e-mails.



o primeiro a apodrecer: após 800K tokens, modelos de ponta deixam passar até 30x mais ações perigosas (arXiv, mai/2026) →

Escrever

persistir fora da janela: notas, memória, AGENTS.md

Selecionar

só o relevante: RAG, poucas tools por etapa

Comprimir

resumir histórico, podar saídas longas

Isolar

subagentes com contexto limpo

O que vimos apodrecer

- Lição esquecida depois de compactar o contexto
- "Bug corrigido no módulo anterior" que nunca existiu
- 2 de 5 juízes LLM herdando o mesmo falso alarme de um contexto impreciso

No case: orçamento por seção do contexto, system prompt intocável, histórico resumido continuamente.

Avalie o avaliador antes de acreditar no número

Quase custou: trocar de modelo por um "overfitting" que era bug do harness

88,9% → 100%

O "overfitting" era o harness comparando com == e diferenciando maiúsculas.

3 de 3

retreinos erraram igual: R\$ 1.870,50 virou 187050. Reprodutível, não ruído: a causa estava no dataset.

1 de 33

agentes avaliadores mudou de veredito num rerun idêntico, sem nada ter mudado.

O que mudou: repetir antes de concluir, tratar $N \leq 3$ como provisório, olhar a distribuição do dataset (só 30,6% dos exemplos tinham centavos) e testar o harness com casos conhecidos.

O workflow gerado que abria a porta do repositório

Quase custou: token de escrita exposto a código de qualquer fork

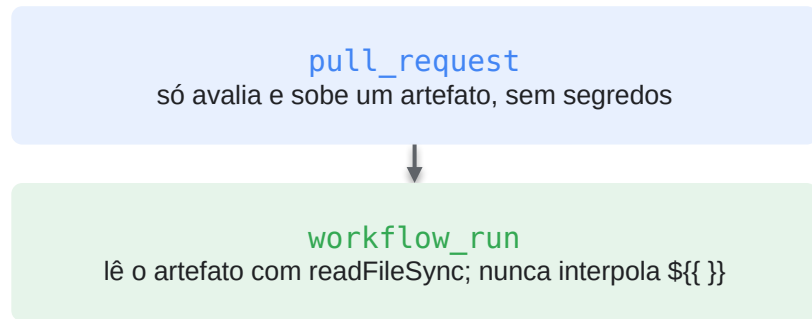
Aconteceu de verdade: Nx (ago/2025): pull_request_target ecoando o título do PR; o malware tentou usar Claude e Gemini locais.

O bot de governança dos PRs quebrava em forks (token só leitura, 403). A "correção" óbvia, a que um agente costuma sugerir, é rodar com permissão de escrita no contexto do PR: o clássico **pwn request**.

```
# perigoso: token de escrita + entrada do fork no shell
on: pull_request_target
run: echo "${{ github.event.pull_request.title }}"
```

Verificado com um PR de fork real.

Como ficou



Vibe coding acelera os 90% fáceis. Os 10% de segurança exigem alguém que saiba ler o que foi gerado.

O que mudou nos times

Habilidades que viraram não negociáveis e práticas que só valorizamos depois.

Habilidades técnicas que viraram não negociáveis

1 Context engineering

Decidir o que entra, sai e persiste em cada volta.

← erro 5

2 Evals e estatística básica

Golden set, rodadas repetidas, desconfiar de N pequeno.

← erro 6

3 Observabilidade

Ler trilhas de agente como se lê log de produção.

← erros 1, 2

4 Segurança agêntica

Prompt injection, Excessive Agency, supply chain de tools e MCP.

← erros 1, 7

5 Ler código gerado

Revisar o que o agente escreveu, sobretudo infra e CI.

← erro 7

6 Contratos e estado

Schemas, Structured Output, limites no código.

← erros 1, 3

Prompt bem escrito deixou de ser diferencial. Virou o mínimo.

Práticas que só aprendemos a valorizar depois

01

Spec antes do prompt

Spec-driven development: o agente implementa um contrato, não uma vibe.

02

Memória do time versionada

Um AGENTS.md (ou CLAUDE.md) espelhado pra todas as ferramentas.

03

Golden set desde o dia 1

Com as frases reais que o modelo produz, não as que imaginamos.

04

HITL proporcional ao risco

Humano onde o erro é caro; automação onde é barato.

05

Conferir o artefato real

Fatura, spec do job, arquivo gerado. Nunca só o resumo.

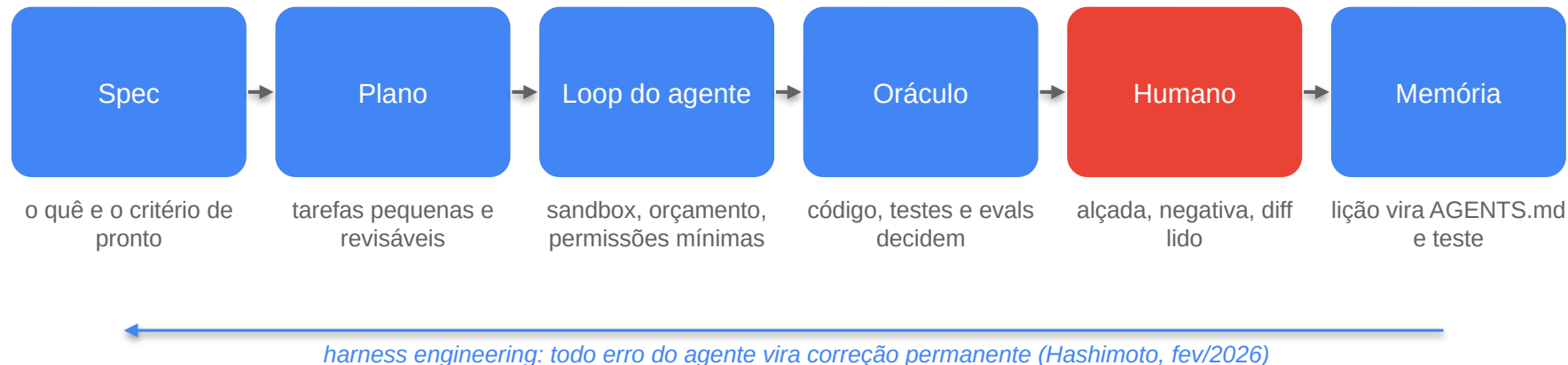
06

Post-mortem de agente

Toda falha vira teste de regressão e linha na memória do time.

Nenhuma é nova. Todas pareciam burocracia até o agente falhar.

O playbook: um loop com contrato



“O loop é o motor. Contexto, verificação e limites são o resto do carro. E quem dirige ainda é o time.”

“Se pensarmos no caminho para a AGI (Inteligência Artificial Geral), então obviamente precisamos de um sistema que consiga raciocinar, dividir problemas e executar ações no mundo.”

Demis Hassabis
(Google DeepMind)

Obrigado!



Ahirton Lopes, PhD. (Ele/Dele)
Senior Manager



LinkedIn



GitHub



<https://www.linkedin.com/in/ahirtonlopes>
<https://github.com/ahirtonlopes>
<https://x.com/ahirtonlopes>

Material



Ahirton Lopes, PhD. (Ele/Dele)
Senior Manager



Slides + código da demo

github.com/ahirtonlopes/alem-dos-loops